

DART PROGRAMMING LANGUAGE

Complete Learning Reference & Code Guide

Topics Covered

- Dart Syntax & Variables
- Data Types & Operators
 - Control Flow & Loops
- Functions & Arrow Syntax
- Collections: Lists, Sets, Maps
- Object-Oriented Programming
 - Null Safety
- Async Programming & Futures
 - Streams
- Error Handling

Version 1.0 • Dart 3.x • Null Safety Enabled

1. Variables & Data Types

Dart is a statically typed language with type inference. Variables can be declared using `var`, `final`, `const`, or explicit type names. Understanding when to use each is fundamental to writing clean Dart code.

1.1 Variable Declaration Keywords

Keyword	Behavior
<code>var</code>	Inferred type, mutable — type set at first assignment
<code>final</code>	Inferred or explicit type, assigned once at runtime
<code>const</code>	Compile-time constant — must be known before program runs
<code>late</code>	Non-nullable variable initialized after declaration
<code>dynamic</code>	Bypasses type checking — use sparingly

```
// var - type inferred from assignment
var name = "Alice";           // String
var age  = 30;                // int
var pi   = 3.14159;           // double

// final - set once, at runtime
final String city = "Mumbai";
final currentTime = DateTime.now(); // OK - runtime value

// const - compile-time constant
const double gravity = 9.8;
const int maxItems  = 100;

// late - initialized before first use
late String description;
description = "Initialized later";
print(description);           // "Initialized later"

// Explicit type annotation
String language = "Dart";
int    version  = 3;
bool   isNullSafe = true;
```

1.2 Primitive Data Types

Type	Description & Examples
<code>int</code>	Whole numbers: 42, -7, 0

Type	Description & Examples
double	Floating-point: 3.14, -0.5, 1.0e10
num	Parent of int & double — accepts either
String	UTF-16 text: "hello", 'world'
bool	true or false only
Null	Represents absence of value — type of null

```
int    score    = 95;
double average  = 87.5;
num    flexible = 42;           // can later hold 42.5
String greeting = "Hello, Dart!";
bool   isActive = true;

// String features
String multiLine = '''
Line one
Line two''';

String interpolated = "Score: $score, Average: ${average.toStringAsFixed(1)}";
print(interpolated); // Score: 95, Average: 87.5

// Type checking and casting
print(score.runtimeType); // int
print(3.14 is double);    // true
print((3.7).toInt());     // 3 (truncates)
print(7.toString());      // "7"
print(int.parse("42"));   // 42
```

Key Rule

Prefer const over final, and final over var. Use explicit types when it improves readability. Avoid dynamic unless absolutely necessary.

2. Operators

Dart supports a rich set of operators including arithmetic, relational, logical, bitwise, assignment, and several null-aware operators unique to Dart.

2.1 Arithmetic Operators

```
int a = 17, b = 5;
```

```
print(a + b);    // 22    - addition
print(a - b);    // 12    - subtraction
print(a * b);    // 85    - multiplication
print(a / b);    // 3.4   - division (always double)
print(a ~/ b);   // 3     - integer division (truncate)
print(a % b);    // 2     - modulo (remainder)
print(a.abs());  // 17    - absolute value (method, not operator)

// Increment / Decrement
int x = 10;
print(x++);      // 10    (post-increment: returns then adds)
print(x);        // 11
print(++x);      // 12    (pre-increment: adds then returns)
```

2.2 Comparison & Logical Operators

```
// Comparison - always returns bool
print(5 == 5);    // true   equal
print(5 != 3);    // true   not equal
print(7 > 3);     // true   greater than
print(3 < 7);     // true   less than
print(5 >= 5);    // true   greater than or equal
print(4 <= 5);    // true   less than or equal

// Logical
bool p = true, q = false;
print(p && q);     // false  AND
print(p || q);    // true   OR
print(!p);        // false  NOT

// Identity
var list1 = [1, 2];
var list2 = list1;
print(identical(list1, list2)); // true - same object
```

2.3 Null-Aware & Cascade Operators

```
// ?? - null coalescing (use right if left is null)
String? name;
print(name ?? "Guest");    // "Guest"

// ??= - assign only if currently null
name ??= "Default";
print(name);               // "Default"

// ?. - null-safe method/property call
String? city;
print(city?.toUpperCase()); // null (no crash)
```

```
// ! - null assertion (throws if null - use carefully)
String? value = "Dart";
print(value!.length);      // 4

// .. - cascade operator (chain calls on same object)
var buffer = StringBuffer()
  ..write("Hello")
  ..write(", ")
  ..write("World!");
print(buffer.toString()); // "Hello, World!"
```

3. Control Flow

Dart provides familiar control flow structures. The switch statement in Dart 3 gained powerful pattern matching capabilities.

3.1 if / else if / else

```
int score = 78;

if (score >= 90) {
  print("A - Excellent");
} else if (score >= 80) {
  print("B - Good");
} else if (score >= 70) {
  print("C - Average");
} else {
  print("F - Needs Improvement");
}
// Output: C - Average

// Ternary operator
String status = score >= 50 ? "Pass" : "Fail";
print(status); // Pass

// if-case (Dart 3)
Object obj = 42;
if (obj case int n when n > 0) {
  print("Positive int: $n");
}
```

3.2 switch Statement & Expressions

```
// Classic switch (Dart 2 style)
String day = "Monday";
switch (day) {
  case "Monday":
  case "Tuesday":
    print("Early week");
    break;
  case "Friday":
    print("TGIF!");
    break;
  default:
    print("Midweek");
}

// Switch expression (Dart 3 - concise & exhaustive)
int statusCode = 404;
String message = switch (statusCode) {
  200 => "OK",
  201 => "Created",
  400 => "Bad Request",
  401 => "Unauthorized",
  404 => "Not Found",
  500 => "Internal Server Error",
  _   => "Unknown Status",
};
print(message); // Not Found
```

3.3 Loops

```
// for loop
for (int i = 0; i < 5; i++) {
  print("i = $i");
}

// for-in loop (iterables)
var fruits = ["apple", "banana", "cherry"];
for (var fruit in fruits) {
  print(fruit);
}

// while loop
int count = 0;
while (count < 3) {
  print("count: $count");
  count++;
}

// do-while (executes at least once)
int n = 5;
do {
  print("n = $n");
}
```

```
n--;  
} while (n > 3);  
  
// forEach (functional style)  
fruits.forEach((f) => print(f.toUpperCase()));  
  
// break and continue  
for (int i = 0; i < 10; i++) {  
  if (i == 3) continue; // skip 3  
  if (i == 7) break;    // stop at 7  
  print(i);            // 0 1 2 4 5 6  
}
```

4. Functions

Functions are first-class objects in Dart. They can be assigned to variables, passed as arguments, and returned from other functions.

4.1 Function Syntax

```
// Standard function  
int add(int a, int b) {  
  return a + b;  
}  
  
// Arrow function (single expression)  
int multiply(int a, int b) => a * b;  
  
// void function (no return value)  
void greet(String name) => print("Hello, $name!");  
  
// Named parameters (with {} – optional by default)  
void createUser({required String name, int age = 18}) {  
  print("User: $name, Age: $age");  
}  
createUser(name: "Bob"); // User: Bob, Age: 18  
createUser(name: "Alice", age: 25); // User: Alice, Age: 25  
  
// Positional optional parameters (with [])  
String greetUser(String name, [String? title]) {  
  return title != null ? "$title $name" : name;  
}  
print(greetUser("Smith", "Dr.)); // Dr. Smith  
print(greetUser("Jones")); // Jones
```

4.2 Higher-Order Functions & Closures

```
// Function as a variable
var square = (int x) => x * x;
print(square(5));           // 25

// Function as parameter
void applyTwice(int x, int Function(int) f) {
  print(f(f(x)));
}
applyTwice(3, square);      // 81  (3^2 = 9, 9^2 = 81)

// Returning a function (closure)
Function makeMultiplier(int factor) {
  return (int x) => x * factor; // captures factor
}
var triple = makeMultiplier(3);
print(triple(7));           // 21

// Anonymous function (lambda)
var numbers = [1, 2, 3, 4, 5];
var doubled = numbers.map((n) => n * 2).toList();
print(doubled);             // [2, 4, 6, 8, 10]

// Recursive function
int factorial(int n) => n <= 1 ? 1 : n * factorial(n - 1);
print(factorial(6));        // 720
```

5. Collections

Dart provides three core collection types: List (ordered, indexed), Set (unordered, unique), and Map (key-value pairs). All support generics for type safety.

5.1 Lists

```
// Fixed-length list
var fixed = List.filled(3, 0);    // [0, 0, 0]

// Growable list (most common)
List<String> fruits = ["apple", "banana", "cherry"];

// List operations
fruits.add("date");               // add to end
fruits.addAll(["elderberry", "fig"]);
fruits.insert(1, "blueberry");    // insert at index
fruits.remove("banana");          // remove by value
```

```
fruits.removeAt(0);           // remove by index
print(fruits.length);        // 5
print(fruits.contains("fig")); // true
print(fruits.indexOf("cherry")); // 1

// Functional methods
var nums = [1, 2, 3, 4, 5, 6, 7, 8];
var evens = nums.where((n) => n.isEven).toList(); // [2,4,6,8]
var squares = nums.map((n) => n * n).toList();    // [1,4,9,16,25,36,49,64]
var sum      = nums.reduce((a, b) => a + b);      // 36
var total    = nums.fold(100, (acc, n) => acc + n); // 136
var sorted   = [...nums]..sort((a, b) => b.compareTo(a)); // descending

// Spread operator and collection-if / collection-for
var extra = [9, 10];
var combined = [...nums, ...extra]; // merge lists

bool addBonus = true;
var result = [1, 2, 3, if (addBonus) 99]; // [1, 2, 3, 99]

var squared = [for (var n in [1,2,3,4]) n * n]; // [1, 4, 9, 16]
```

5.2 Sets

```
// Sets contain only unique values
Set<int> primes = {2, 3, 5, 7, 11};

primes.add(13);
primes.add(7);           // ignored - already exists
print(primes.length);    // 6

// Set operations
var a = {1, 2, 3, 4};
var b = {3, 4, 5, 6};
print(a.union(b));       // {1, 2, 3, 4, 5, 6}
print(a.intersection(b)); // {3, 4}
print(a.difference(b));   // {1, 2}
print(a.containsAll({1, 2})); // true

// Convert list to set (remove duplicates)
var dupes = [1, 2, 2, 3, 3, 4];
var unique = dupes.toSet().toList(); // [1, 2, 3, 4]
```

5.3 Maps

```
Map<String, int> scores = {
  "Alice": 95,
```

```
"Bob": 87,
"Carol": 92,
};

// Access and modify
scores["Dave"] = 88;           // add entry
scores["Alice"] = 97;          // update entry
scores.remove("Bob");          // remove entry

print(scores["Carol"]);        // 92
print(scores["Eve"]);          // null (key not found)
print(scores.containsKey("Dave")); // true
print(scores.containsValue(92)); // true

// Iteration
scores.forEach((name, score) => print("$name: $score"));
print(scores.keys.toList());   // [Alice, Carol, Dave]
print(scores.values.toList()); // [97, 92, 88]

// Map.from and putIfAbsent
var copy = Map<String, int>.from(scores);
copy.putIfAbsent("Eve", () => 75); // only adds if missing

// Map with complex values
Map<String, List<String>> courses = {
  "CS101": ["Alice", "Bob"],
  "MATH": ["Carol", "Dave"],
};
courses["CS101"]?.add("Eve");
```

6. Object-Oriented Programming

Dart is a fully object-oriented language. Everything is an object, including numbers and functions. Dart supports classes, inheritance, mixins, and interfaces.

6.1 Classes & Constructors

```
class Animal {
  // Instance variables
  String name;
  int age;
  String _species; // private (underscore prefix)

  // Default constructor
  Animal(this.name, this.age, this._species);
}
```

```
// Named constructor
Animal.unknown() : name = "Unknown", age = 0, _species = "Unknown";

// Named constructor with initializer list
Animal.puppy(String name) : this(name, 0, "Canis lupus");

// Getter
String get species => _species;

// Setter with validation
set animalAge(int value) {
  if (value >= 0) _species = _species; // placeholder
  age = value >= 0 ? value : 0;
}

// Instance method
void speak() => print("$name says: ...");

// toString override
@override
String toString() => "Animal($name, age: $age, $_species)";
}

var cat = Animal("Whiskers", 3, "Felis catus");
var dog = Animal.puppy("Rex");
print(cat.species); // Felis catus
print(dog); // Animal(Rex, age: 0, Canis lupus)
```

6.2 Inheritance

```
class Dog extends Animal {
  String breed;

  // super constructor call
  Dog(String name, int age, this.breed)
    : super(name, age, "Canis lupus familiaris");

  @override
  void speak() => print("$name barks: Woof!");

  void fetch() => print("$name fetches the ball!");

  @override
  String toString() => "Dog($name, breed: $breed)";
}

class GuideDog extends Dog {
  String owner;

  GuideDog(String name, String breed, this.owner)
```

```
        : super(name, 3, breed);

@override
void speak() {
    super.speak();    // call parent
    print("(guide dog for $owner)");
}
}

var guide = GuideDog("Buddy", "Labrador", "John");
guide.speak();
// Buddy barks: Woof!
// (guide dog for John)
print(guide is Dog);    // true
print(guide is Animal); // true
```

6.3 Abstract Classes & Interfaces

```
// Abstract class – cannot be instantiated directly
abstract class Shape {
    double get area;        // abstract getter
    double get perimeter;   // abstract getter
    void describe() => print("I am a ${runtimeType} with area
    ${area.toStringAsFixed(2)}");
}

// Dart uses "implements" for interface-style contracts
class Circle extends Shape {
    final double radius;
    Circle(this.radius);

    @override double get area        => 3.14159 * radius * radius;
    @override double get perimeter => 2 * 3.14159 * radius;
}

class Rectangle extends Shape {
    final double width, height;
    Rectangle(this.width, this.height);

    @override double get area        => width * height;
    @override double get perimeter => 2 * (width + height);
}

List<Shape> shapes = [Circle(5), Rectangle(4, 6)];
for (var s in shapes) s.describe();
// I am a Circle with area 78.54
// I am a Rectangle with area 24.00
```

6.4 Mixins

```
// Mixins add reusable behavior without inheritance chains
mixin Flyable {
  void fly() => print("${toString()} is flying!");
  int get maxAltitude => 10000;
}

mixin Swimmable {
  void swim() => print("${toString()} is swimming!");
  int get maxDepth => 50;
}

class Bird {
  String name;
  Bird(this.name);
  @override String toString() => name;
}

// Combine multiple mixins with "with"
class Duck extends Bird with Flyable, Swimmable {
  Duck(String name) : super(name);
}

var duck = Duck("Donald");
duck.fly();           // Donald is flying!
duck.swim();          // Donald is swimming!
print(duck.maxAltitude); // 10000
print(duck is Flyable);  // true
print(duck is Swimmable); // true
```

6.5 Factory Constructors

```
class Logger {
  static final Logger _instance = Logger._internal();
  final List<String> logs = [];

  // Private constructor
  Logger._internal();

  // Factory returns the same instance (singleton)
  factory Logger() => _instance;

  void log(String message) {
    logs.add("[LOG] $message");
    print("[LOG] $message");
  }
}

var log1 = Logger();
var log2 = Logger();
log1.log("App started");
log2.log("User logged in");
```

```
print(identical(log1, log2)); // true - same object
print(log1.logs.length);    // 2
```

7. Null Safety

Dart's sound null safety (introduced in Dart 2.12) eliminates null reference errors at compile time. Every type is non-nullable by default.

Sound Null Safety

In Dart, `String` is guaranteed non-null. `String?` can be null. The compiler enforces this — null errors become compile-time mistakes, not runtime crashes.

7.1 Nullable vs Non-Nullable

```
// Non-nullable - MUST have a value
String name = "Dart";    // OK
// String name = null;   // COMPILER ERROR

// Nullable - can be null (add ?)
String? nickname;        // null by default
nickname = "Flutter Dev"; // OK
nickname = null;          // also OK

// int vs int?
int score = 0;            // must be set
int? optionalScore;       // can be null
```

7.2 Null-Aware Operators

```
String? email;

// ?. - safe navigation (returns null if receiver is null)
print(email?.toUpperCase()); // null (no NullPointerException)
print(email?.length);        // null

// ?? - null coalescing (provide fallback)
String display = email ?? "no-reply@example.com";
print(display);             // no-reply@example.com

// ??= - assign if null
email ??= "user@example.com";
```

```
print(email); // user@example.com

// ! - null assertion operator (use with caution!)
String? value = "Hello";
int len = value!.length; // 5 - asserts non-null
// If value were null, this throws at runtime

// Chaining null-aware operators
Map<String, String?>? user;
print(user?["email"]?.toUpperCase()); // null - no crash
```

7.3 Late Keyword

```
// late - tells Dart "I promise this will be set before use"
class DatabaseService {
  late String connectionString; // non-nullable but set later

  void init(String host) {
    connectionString = "postgres://$host/mydb";
  }

  void query(String sql) {
    // connectionString must be set before calling query()
    print("Query on $connectionString: $sql");
  }
}

// late final - lazy initialization
class ExpensiveCalculator {
  late final int result = _compute();

  int _compute() {
    print("Computing..."); // only runs once
    return 42;
  }
}

var calc = ExpensiveCalculator();
print(calc.result); // Computing... 42
print(calc.result); // 42 (cached - not recomputed)
```

7.4 Null Checks & Promotion

```
String? maybeString = getSomeString();

// Type promotion - Dart narrows type after null check
if (maybeString != null) {
```

```
// Inside here, Dart knows maybeString is String (not String?)
print(maybeString.toUpperCase()); // no ? needed
print(maybeString.length);        // safe
}

// Using is for pattern matching with promotion
Object? obj = 42;
if (obj is int) {
  print(obj + 10); // obj promoted to int here
}

// Required keyword for named params
void createProfile({required String name, String? bio}) {
  print("Name: $name, Bio: ${bio ?? "N/A"}");
}
createProfile(name: "Alice"); // OK
// createProfile();           // COMPILER ERROR

String? getSomeString() => "Hello";
```

8. Async Programming

Dart uses an event loop model. The `async/await` syntax makes asynchronous code readable. Futures represent values that will be available later; Streams represent sequences of async values.

8.1 Futures & `async/await`

```
// A Future represents a single async value
Future<String> fetchUserName(int id) async {
  // Simulate network delay
  await Future.delayed(Duration(seconds: 1));
  return "User_$id";
}

// Using await — must be inside an async function
Future<void> main() async {
  print("Fetching user...");
  String name = await fetchUserName(42);
  print("Got: $name"); // Got: User_42
}

// Future chaining with .then()
fetchUserName(1)
  .then((name) => print("Name: $name"))
  .catchError((e) => print("Error: $e"));
```

```
// Future.wait - run multiple futures concurrently
Future<void> loadDashboard() async {
  var results = await Future.wait([
    fetchUserName(1),
    fetchUserName(2),
    fetchUserName(3),
  ]);
  print(results); // [User_1, User_2, User_3] - all done
}
```

8.2 Error Handling

```
Future<int> divide(int a, int b) async {
  if (b == 0) throw ArgumentError("Cannot divide by zero");
  await Future.delayed(Duration(milliseconds: 100));
  return a ~/ b;
}

// try-catch-finally with async
Future<void> safeDivide() async {
  try {
    int result = await divide(10, 0);
    print("Result: $result");
  } on ArgumentError catch (e) {
    print("Argument error: ${e.message}"); // caught!
  } on Exception catch (e) {
    print("General exception: $e");
  } catch (e, stackTrace) {
    print("Unknown error: $e");
    print(stackTrace);
  } finally {
    print("Cleanup done - always runs");
  }
}

// Custom exceptions
class NetworkException implements Exception {
  final String message;
  final int statusCode;
  NetworkException(this.message, this.statusCode);
  @override String toString() => "NetworkException $statusCode: $message";
}

Future<String> fetchData(String url) async {
  // Simulated error
  throw NetworkException("Server unavailable", 503);
}
```

8.3 Streams

```
// Stream - sequence of async values over time

// Creating a simple stream with async*
Stream<int> countdown(int from) async* {
  for (int i = from; i >= 0; i--) {
    await Future.delayed(Duration(milliseconds: 500));
    yield i;    // emit each value
  }
}

// Listening to a stream
Future<void> main() async {
  await for (int n in countdown(5)) {
    print(n);    // 5, 4, 3, 2, 1, 0
  }
  print("Blast off!");
}

// StreamController for manual stream control
import "dart:async";

var controller = StreamController<String>();
controller.stream
  .where((msg) => msg.startsWith("ERROR"))
  .listen((msg) => print("Alert: $msg"));

controller.add("INFO: App started");
controller.add("ERROR: Disk full");    // triggers listener
controller.add("DEBUG: Checkpoint");
controller.close();

// Stream transformations
Stream<int> numbers = Stream.fromIterable([1,2,3,4,5,6,7,8]);
numbers
  .where((n)  => n.isEven)
  .map((n)    => n * n)
  .listen((n) => print(n)); // 4, 16, 36, 64
```

9. Practice Programs

The following examples demonstrate practical Dart programs combining all the concepts covered in this guide.

9.1 Calculator Class

```
class Calculator {
  double _memory = 0;

  double add(double a, double b) => a + b;
  double subtract(double a, double b) => a - b;
  double multiply(double a, double b) => a * b;
  double divide(double a, double b) {
    if (b == 0) throw ArgumentError("Division by zero");
    return a / b;
  }

  void storeMemory(double val) => _memory = val;
  double recallMemory() => _memory;
  void clearMemory() => _memory = 0;

  double power(double base, int exp) =>
    List.generate(exp, (_) => base).fold(1.0, (a, b) => a * b);

  double percentage(double val, double pct) => val * pct / 100;
}

void main() {
  var calc = Calculator();
  print(calc.add(15, 27));           // 42.0
  print(calc.divide(100, 4));        // 25.0
  print(calc.power(2, 8));           // 256.0
  print(calc.percentage(200, 15));   // 30.0
  calc.storeMemory(99);
  print(calc.recallMemory());        // 99.0
}
```

9.2 Todo List with Filtering

```
enum Priority { low, medium, high }

class Todo {
  final String id;
  String title;
  bool isCompleted;
  Priority priority;
  final DateTime createdAt;

  Todo({
    required this.title,
    this.isCompleted = false,
    this.priority = Priority.medium,
  }) : id = DateTime.now().millisecondsSinceEpoch.toString(),
      createdAt = DateTime.now();

  void complete() => isCompleted = true;
  @override String toString() =>
```

```
    "[${isCompleted ? "x" : " "}] $title (${priority.name})";
  }

class TodoList {
  final List<Todo> _todos = [];

  void add(Todo todo) => _todos.add(todo);
  void remove(String id) => _todos.removeWhere((t) => t.id == id);
  void complete(String id) => _todos.firstWhere((t) => t.id == id).complete();

  List<Todo> get pending => _todos.where((t) => !t.isCompleted).toList();
  List<Todo> get completed => _todos.where((t) => t.isCompleted).toList();
  List<Todo> byPriority(Priority p) => _todos.where((t) => t.priority ==
p).toList();

  void printAll() => _todos.forEach(print);
}
```

9.3 Async API Simulation

```
class ApiService {
  static final _db = <int, Map<String, dynamic>>{
    1: {"name": "Alice", "role": "Admin"},
    2: {"name": "Bob", "role": "User"},
    3: {"name": "Carol", "role": "Editor"},
  };

  Future<Map<String, dynamic>> getUser(int id) async {
    await Future.delayed(Duration(milliseconds: 300));
    final user = _db[id];
    if (user == null) throw Exception("User $id not found");
    return user;
  }

  Future<List<Map<String, dynamic>>> getAllUsers() async {
    await Future.delayed(Duration(milliseconds: 500));
    return _db.values.toList();
  }
}

Future<void> main() async {
  var api = ApiService();

  // Single user
  try {
    var user = await api.getUser(1);
    print("Fetched: $user");
    var missing = await api.getUser(99); // throws
  } catch (e) {
    print("Error: $e");
  }
}
```

```
// All users concurrently
var ids = [1, 2, 3];
var users = await Future.wait(ids.map(api.getUser));
users.forEach(print);
}
```

10. Quick Reference Cheatsheet

10.1 Type System

Type	Meaning
String	Non-nullable string — must be assigned
String?	Nullable string — can be null
int	Integer numbers
double	Floating-point numbers
bool	true or false
List<T>	Ordered collection of type T
Set<T>	Unordered unique values of type T
Map<K,V>	Key-value pairs
dynamic	Opt-out of static typing — avoid
Object	Root of all non-nullable types
Object?	Root of all types including null

10.2 OOP Keywords

Keyword	Purpose
extends	Inherit from a class (single inheritance)
implements	Fulfill a contract — must implement all members
with	Apply one or more mixins
abstract	Class that cannot be instantiated directly
mixin	Reusable behavior unit — no constructor
static	Class-level member — not on instances

Keyword	Purpose
<code>@override</code>	Annotation to mark intentional overrides
<code>super</code>	Reference parent class constructor/methods
<code>this</code>	Reference current instance
<code>factory</code>	Constructor that controls what is returned

10.3 Async Keywords

Keyword/Type	Purpose
<code>async</code>	Marks a function as asynchronous — returns <code>Future</code>
<code>await</code>	Pauses execution until <code>Future</code> completes
<code>async*</code>	Marks a generator function — returns <code>Stream</code>
<code>yield</code>	Emits a value from a generator (<code>async*</code> or <code>sync*</code>)
<code>sync*</code>	Synchronous generator — returns <code>Iterable</code>
<code>Future</code>	Represents a single pending async result
<code>Stream</code>	Represents a sequence of async events